

Easy Tags

(V1.0)

An asset for Unity to extend the functionality of tags

Documentation

Author:

Ferhat Cemoglu (FeroDev)

26/06/2024

Table of Contents

1: Tags.....	3
1.1 Accessible Tags.....	3
1.2 Tag Attribute.....	5
2: Multiple Tags.....	6
2.1 Tag Mask Property	6
2.2 Multiple Tags Component.....	6
2.3 Built-In Extension Functions.....	8
2.3.1 TagMask Property Extension Functions	8
2.3.2 MultipleTags Component Extension Functions:	9
3: TagMaskUtils	10
3.1 Example Usage	10
3.2 TagMaskUtils Functions	14
3.2.1 Getting Selected Tags Functions	14
3.2.2 Finding Game Object Functions.....	14
3.2.3 Compare Tag Functions.....	15
3.2.4 Has With Functions	16
3.2.5 Has Matching Functions.....	17
3.2.6 Contains From Functions	17
3.2.7 Find Transform Functions.....	18
3.2.8 Updating TagMask Functions	19
3.2.9 Finding Game Objects and Transforms without MultipleTags.....	20
5: References.....	21

1: Tags

1.1 Accessible Tags

In location: “Assets/Cyafen Studios/Easy Tags/Scripts/Tags” you will find a script called “Tags.cs”. This class contains string constants for each of the defined tags in Unity. This lets you to access tags by their defined names. It is safer and prevents you to use undefined tags. You can also be sure that a tag is not defined without going to the play mode.

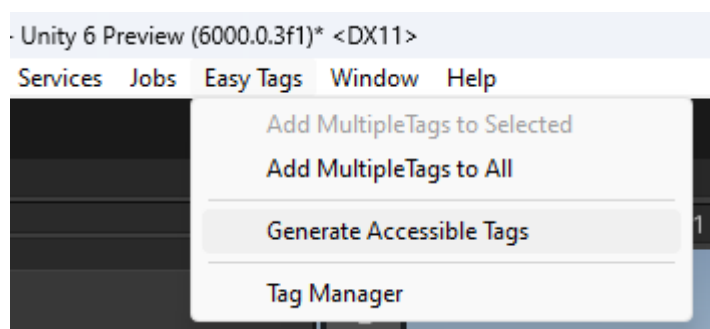
Instead of this:

```
0 references
private void GetPlayer()
{
    ...
    GameObject player = GameObject.FindGameObjectWithTag("Player");
    ...
}
```

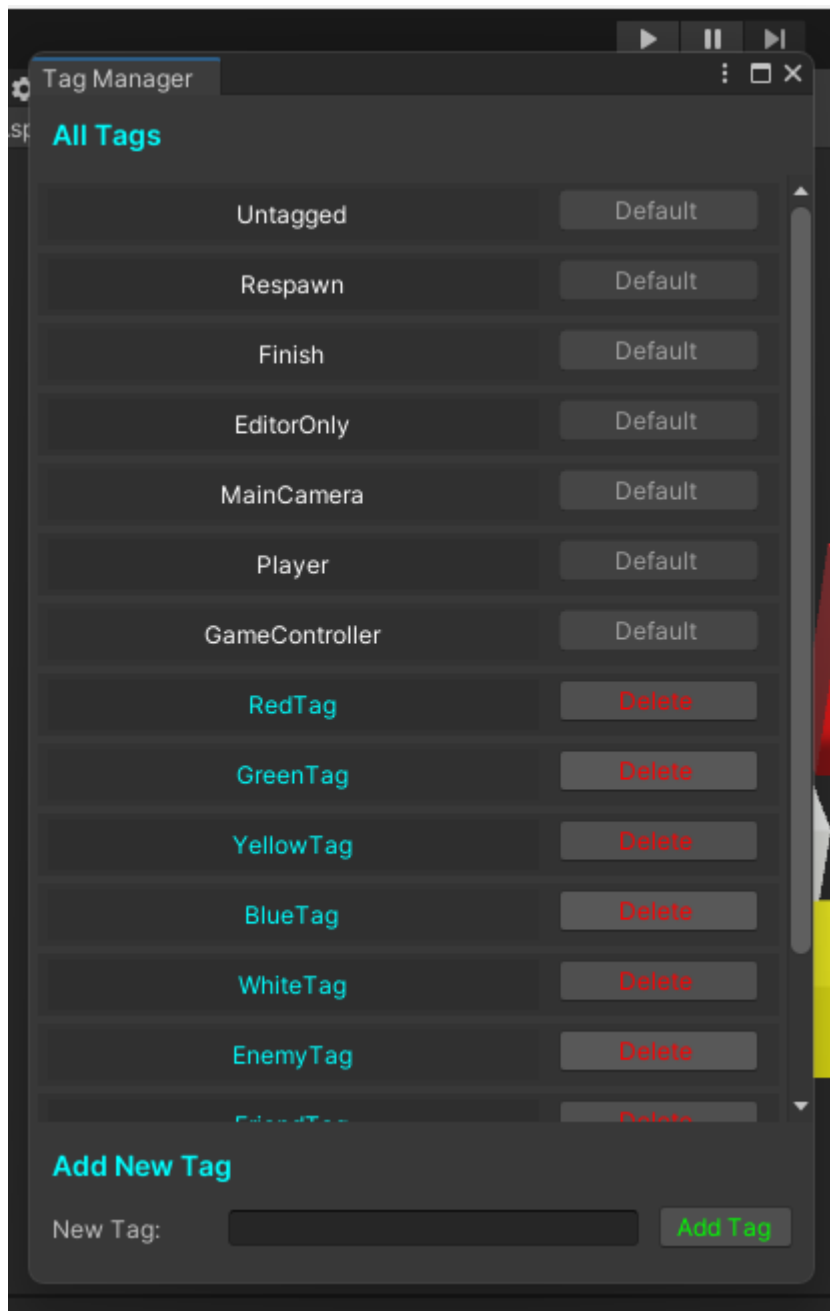
You can do this:

```
0 references
private void GetPlayer()
{
    ...
    GameObject player = GameObject.FindGameObjectWithTag(Tags.Player);
    ...
}
```

“Tags.cs” is already present in the package and automatically gets created during the package import, but if it’s not, or in case of an accidental removal, or there are errors in the console saying that Tags class is not defined, find “Easy Tags” option from the menu at the top left of the screen, and click the “Generate Accessible Tags” button:



The project has to be saved after creating a new tag to update the accessible tags. There is an improved tag management system which lets you to update the accessible tags without saving the project. From the same menu, click “[Tag Manager](#)” button:



Use this window to add a new tag and delete a user defined tag and update the accessible tags automatically, without the need to save the project.

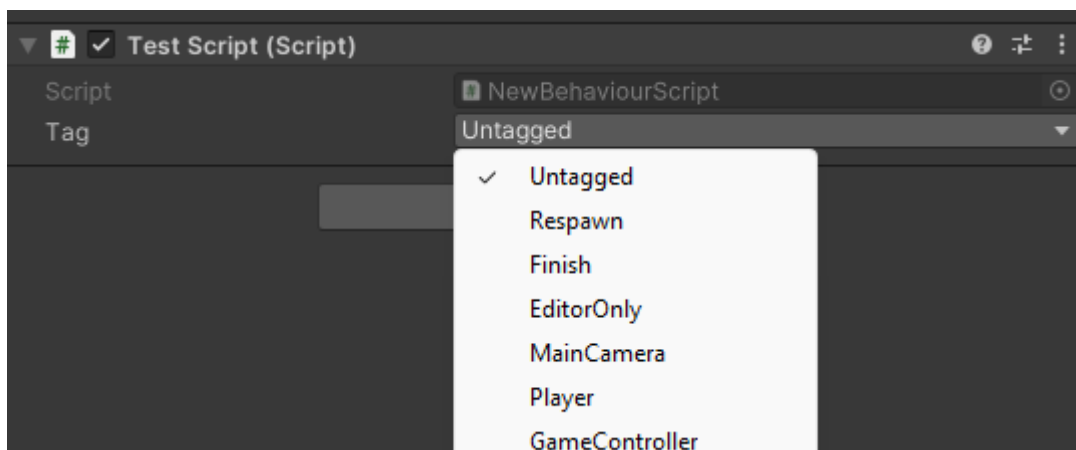
1.2 Tag Attribute

Tag attribute, [Tag], can be used in front of string fields to display a selectable drop-down list of defined tags in the inspector, instead of writing by hand. This is a safer way to use tags as it limits you to pick specific values to a variable.

Add the [Tag] attribute in front of a string field:

```
[Tag] public string Tag;
```

The string field will be displayed as a drop-down menu of tags:



2: Multiple Tags

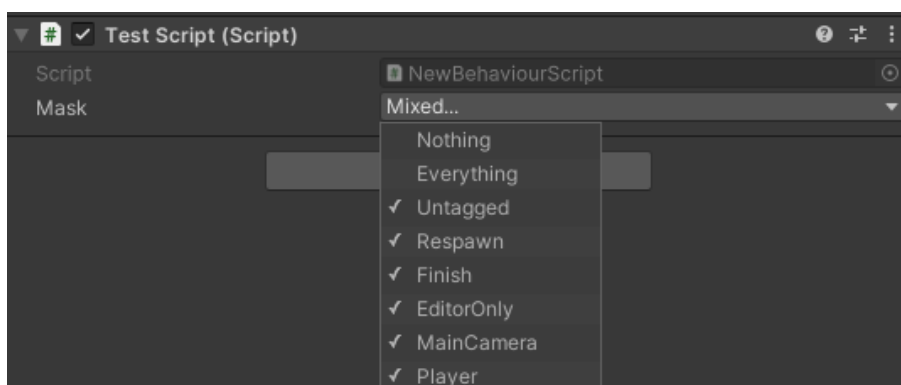
2.1 Tag Mask Property

[TagMask](#) utilizes bitmasks to let you select multiple tags for any situation that you want to use multiple tags for. Bitmasks represent an integer with 32 layers and define the layers as true or false [1].

Use a [TagMask](#) property:

```
public TagMask mask;
```

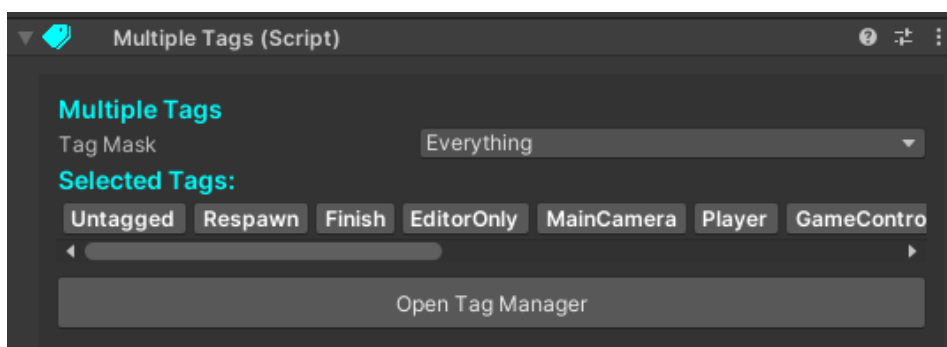
To be able to select multiple tags in the inspector menu:



2.2 Multiple Tags Component

[MultipleTags](#) is a custom component which utilizes [TagMask](#) to let you assign multiple tags to game objects and allows you to use extended utility functions for multiple tags.

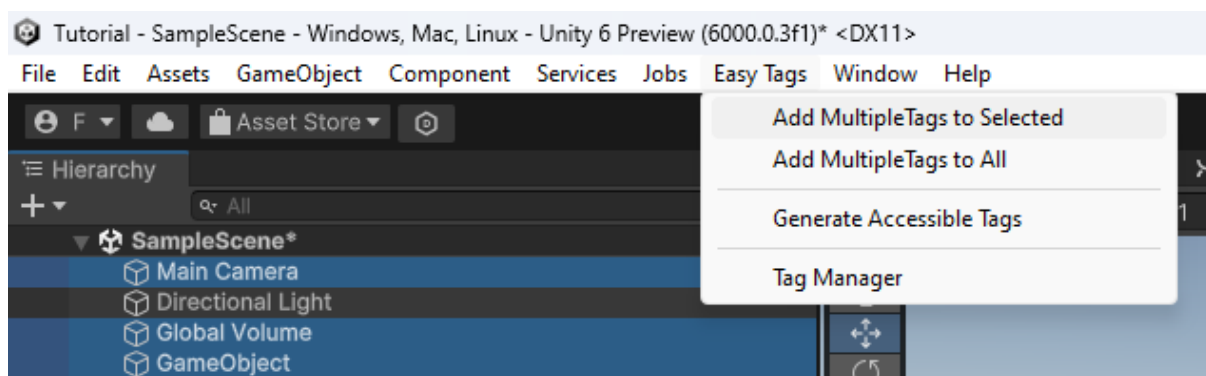
Use [MultipleTags](#) Component to assign multiple tags to game objects:



What you can do:

- Select tags with the [TagMask](#) property inside the [MultipleTags](#) component.
- Click on the selected tags to remove them from the TagMask.
- Open tag manager to manage your defined tags.

You can either add [MultipleTags](#) component using the conventional method for adding any components or scripts. Another way to add [MultipleTags](#) component is to select the game objects from the hierarchy that you want to add [MultipleTags](#) to, and use the same “[Easy Tags](#)” menu and click the “[Add Multiple Tags to Selected](#)” button:



This will automatically add [MultipleTags](#) component to all the selected game objects in the hierarchy. This method will also ensure that whatever tag the selected game object has, is also automatically added to the [TagMask](#) of their [MultipleTags](#) component. Alternatively, you can also add [MultipleTags](#) component to all game objects in the hierarchy using the same menu with “[Add MultipleTags to All](#)” button.

2.3 Built-In Extension Functions

Both [TagMask](#) property and [MultipleTags](#) component have some built-in extension functions to help you with your development

2.3.1 TagMask Property Extension Functions

Public [TagMask](#) (multiple options):

- Constructor to initialize the [TagMask](#)
- Takes either one of the following types:
 - o Integer
 - o String
 - o Array of strings
 - o List of strings

Public string GetSelectedTags ():

- Returns an array of strings consisting of the selected tags in the mask.

Public bool HasTag (multiple options):

- Takes either one of the following types:
 - o Integer
 - o String
- Returns true if the argument given is present in the mask.

Public bool HasTags (multiple options):

- Takes either one of the following types:
 - o Integer
 - o Array of strings
 - o List of strings
- Returns true if the mask matches with another or has all the tags.

Public void AddTagToMask (string):

- Adds a tag to the mask.

Public void AddTagsToTheMask (multiple options):

- Takes either one of the following types:
 - o Array of strings
 - o List of strings
- Adds multiple tags to the mask

Public void RemoveTagFromMask (string):

- Removes a tag from the mask.

Public void RemoveTagsFromMask (multiple options):

- Takes either one of the following types:
 - o Array of strings
 - o List of strings
- Removes multiple tags from the mask.

Public void ToggleTag (string):

- Toggles a tag in the mask.

Public void ToggleTags (multiple options):

- Takes either one of the following types:
 - o Array of strings
 - o List of strings
- Toggles multiple tags in the mask.

Public bool MatchesWith (multiple options):

- Takes either one of the following types:
 - o Integer
 - o TagMask
- Returns true if the masks match (no different tag is allowed)

2.3.2 MultipleTags Component Extension Functions:

[MultipleTags](#) component mainly has similar functions with the same naming and the same parameters. The different functions are as follows:

Public bool ContainsTagFrom (multiple options):

- Takes either one of the following types:
 - o Array of game objects
 - o List of game objects
- Returns true if the self game object has any matching tags with any game objects from the array of list.

Public bool ContainsTagsFrom (multiple options):

- Takes either one of the following types:
 - o Array of game objects
 - o List of game objects
- Returns true if the self game object has all matching tags with any game objects from the array of list.

Public bool CompareTag (GameObject):

- Returns true if self and other game object has any matching tags.

Public bool CompareTags (GameObject):

- Returns true if self and other game object has all tags matching.

3: TagMaskUtils

TagMaskUtils is a static utility class that has over 60 utility functions which you can globally access. The functions have similar naming with the built-in Unity functions that uses tags. These functions are to extend the functionalities of tags by utilizing [MultipleTags](#). Therefore, most of these functions only work with the game objects that has [MultipleTags](#) component attached. Some other functions in this class that utilise [TagMask](#) without [MultipleTags](#) are also added, in case you want to use default Unity tag system with [TagMask](#).

3.1 Example Usage

Suppose you attached [MultipleTags](#) to some of your game objects or prefabs with different combinations. Throughout your game, you want to be able to find some of those game objects (specific type of enemy, for example) and do something with them.

Let's remember how we can find a game object with a tag using the default Unity method:

```

Unity Message | 0 references
void Start()
{
    GameObject gameObject = GameObject.FindGameObjectWithTag("Enemy");
    Debug.Log(gameObject.name);
}

```

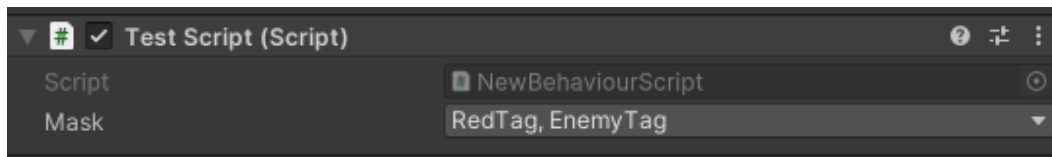
This method only works with single tags, but we have multiple tags on our enemies.

There are two main methods we can use to find our specific type of enemies. The first one is with a [TagMask](#) field predefined:

First, define your [TagMask](#) field:

```
public TagMask mask;
```

Then, choose what tags should be used while searching in the inspector:



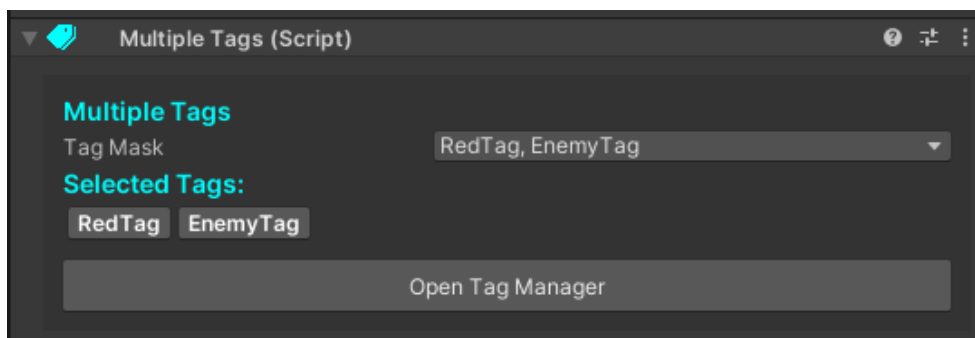
Then we can use this mask to search for a specific type of enemy that has [MultipleTags](#):

```

Unity Message | 0 references
void Start()
{
    GameObject gameObject = TagMaskUtils.FindGameObjectWithTags(mask);
    Debug.Log(gameObject.name);
}

```

This will give us the first game object in the inspector that has [MultipleTags](#) and has all the tags from the mask selected in its [MultipleTags](#):



The other method is that instead of defining a [TagMask](#) field, you can also define a local [TagMask](#) variable:

```

Unity Message | 0 references
void Start()
{
    TagMask tagMask = new TagMask();
    tagMask.AddTagsToMask(new string[] { Tags.RedTag, Tags.EnemyTag});

    GameObject gameObject = TagMaskUtils.FindGameObjectWithTags(tagMask);
    Debug.Log(gameObject.name);
}

```

Note that you can also directly initialize the TagMask with a constructor:

```

TagMask tagMask = new TagMask(new string[] { Tags.RedTag, Tags.EnemyTag });

```

Another usage that can be given as an example is when you want to compare tags. If you want to compare tags while collision handling, using the Unity default method, you are probably doing something like this:

```

Unity Message | 0 references
private void OnCollisionEnter(Collision collision)
{
    if (collision.gameObject.CompareTag("Enemy"))
    {
        TakeDamage(10);
    }
}

```

But now, you can use [MultipleTags](#) and [TagMask](#) to compare for multiple tags:

```

Unity Message | 0 references
private void OnCollisionEnter(Collision collision)
{
    if (TagMaskUtils.CompareTags(collision.gameObject, mask))
    {
        TakeDamage(10);
    }
}

```

Here are other methods that you can write this:

```

Unity Message | 0 references
private void OnCollisionEnter(Collision collision)
{
    if (collision.gameObject.GetComponent<MultipleTags>().HasTags(mask))
    {
        TakeDamage(10);
    }
}

```

```

Unity Message | 0 references
private void OnCollisionEnter(Collision collision)
{
    if (collision.gameObject.TryGetComponent<MultipleTags>(out var multipleTags)) return;

    if (multipleTags.HasTags(new string[] {Tags.EnemyTag, Tags.RedTag}))
    {
        TakeDamage(10);
    }
    else if (multipleTags.HasTags(new string[] { Tags.EnemyTag, Tags.YellowTag }))
    {
        TakeDamage(5);
    }
}

```

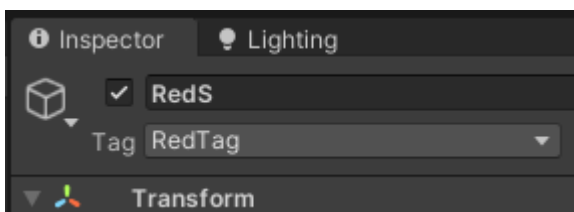
Suppose you want to find game objects tagged with the default Unity tag system, but you don't want to use [MultipleTags](#), and you still want to include more than one tags in your search. You can use these functions from [TagMaskUtils](#) to utilise [TagMask](#) without the [MultipleTags](#) component:

```

Unity Message | 0 references
void Start()
{
    GameObject[] gameObject = TagMaskUtils.FindGameObjectsWithoutMultipleTags(Mask);
}

```

This will return all the game objects that are tagged with any of the tags in the mask using the default Unity tag system, like:



Note that you can also find the first game object instead of all game objects:

```

GameObject gameObject = TagMaskUtils.FindGameObjectWithoutMultipleTags(Mask);

```

3.2 TagMaskUtils Functions

Because there are over 60 utility functions, there is a naming rule which helps you to remember which function takes what arguments and returns with what. The naming rule can be summarised with the following list:

- Getting Selected Tags Functions: Returns the selected tags.
- Finding Game Objects Functions: Find game objects with multiple tags.
- Compare Tag Functions: Compare tags with various ways.
- Has With Functions: Compare tags in an array or a list with multiple tags.
- Has Matching Functions: Compare the tags of elements in an array or a list.
- Contains From Functions: Compare a game object with an array or a list.
- Find Transform Functions: Find transforms with multiple tags.
- Updating [TagMask](#) Functions: Updates the given [TagMask](#).
- Finding Game Objects and Transforms without [MultipleTags](#).

It is also important to remember that TagMask is represented with a 32-layer bitmask, which is an integer type.

3.2.1 Getting Selected Tags Functions

Public static string[] GetSelectedTags (multiple options):

- Takes either:
 - o Integer
 - o TagMask
 - o MultipleTags
- Returns: Array of strings for selected tags.

3.2.2 Finding Game Object Functions

For an overview of the naming rule for these functions is that they are written as:

- FindGameObject(s)WithTag(s)

Where the first (s) determines whether the return is going to be a single game object or an array of game objects. The second (s) determines whether a single tag or all the tags will be used while searching.

Public static GameObject FindGameObjectWithTag (multiple options):

- Takes either:
 - o Integer
 - o TagMask
- Returns: First game object with any of the tags in the mask.

Public static GameObject FindGameObjectsWithTags (multiple options):

- Take either:
 - o Integer
 - o TagMask
- Returns: First game object with all the tags in the mask.

Public static GameObject[] FindGameObjectsWithTag (multiple options);

- Take either:
 - o Integer
 - o TagMask
- Returns: All the game objects with any tags in the mask.

Public GameObject[] FindGameObjectsWithTags (multiple options):

- Takes either:
 - o Integer
 - o TagMask
- Returns: All the game objects with all the tags in the mask.

3.2.3 Compare Tag Functions

Similar with finding game objects, the naming rule looks like:

- CompareTag(s)

Where the (s) determines whether a single tag or all the tags in the mask is used for the comparison. These functions return a Boolean value.

Public static bool CompareTag (multiple options):

- Takes either:
 - o GameObject AND integer
 - o GameObject AND TagMask
 - o GameObject AND GameObject
- Returns true if any matching tags are found.

Public static bool CompareTags (multiple options):

- Takes either:
 - o GameObject, integer
 - o GameObject, TagMask
 - o GameObject, GameObject
- Returns true if all tags are matching (no different tag is allowed).

3.2.4 Has With Functions

The overview of the naming rule for these functions is:

- Has(Any/All)With(Any/All)Tag(s)

Where the first (Any/All) determines whether only one element or all elements from an array or a list is/are getting compared. The second (Any/All) determines whether a single tag or all of the tags are used for comparing. These functions return a Boolean value.

Public static bool HasAnyWithAnyTag (multiple options):

- Takes either:
 - o GameObject[] OR List<GameObject> AND Integer OR TagMask
- Returns:
 - o True if any element has any tag in the mask.

Public static bool HasAllWithAllTags (multiple options):

- Takes either:
 - o GameObject[] OR List<GameObject> AND Integer OR TagMask
- Returns:
 - o True if all elements have all the tags in the mask (no different tag).

Public static bool HasAnyWithAllTags (multiple options):

- Takes either:
 - o GameObject[] OR List<GameObject> AND Integer OR TagMask
- Returns:
 - o True if any element has all the tags in the mask (no different tag).

Public static bool HasAllWithAnyTag(multiple options):

- Takes either:
 - o GameObject[] OR List<GameObject> AND Integer OR TagMask
- Returns:
 - o True if all elements have any tag in the mask (different tags between elements is allowed).

3.2.5 Has Matching Functions

The naming rule is as follows:

- Has(Any/All)Matching(Tag(s))

Where (Any/All) determines whether any element or all of the elements are used for comparing. (Tag(s)) determines whether any tag or all the tags in the mask is used for comparing. These functions return a Boolean value.

Public static bool HasAnyMatchingTags (multiple options):

- Take either:
 - o GameObject[]
 - o List<GameObject>
- Returns true if any element has all matching tags with another (no different tags).

Public static bool HasAnyMatchingTag (multiple options):

- Take either:
 - o GameObject[]
 - o List<GameObject>
- Returns true if any element has any matching tag with another.

Public static bool HasAllMatching (multiple options):

- Take either:
 - o GameObject[]
 - o List<GameObject>
- Returns true if all elements have all matching tags (no different tag).

3.2.6 Contains From Functions

The naming rule for these functions is as follows:

- ContainsTag(s)From

Where the (s) determines whether only one tag or all tags will be used for comparing. These functions return a Boolean value.

Public static bool ContainsTagFrom (multiple options):

- Takes either:
 - o GameObject AND GameObject[]
 - o GameObject AND List<GameObject>
- Returns true if a different game object has any matching tag with any element from an array or a list.

Public static bool ContainsTagsFrom (multiple options):

- Takes either:
 - o GameObject AND GameObject[]
 - o GameObject AND List<GameObject>
- Returns true if a different game object has all matching tags with any element from an array or a list.

3.2.7 Find Transform Functions

These functions are similar to find game object functions but return transform instead. The naming rule is as follows:

- Find(All)WithTag(s)

Where (All) determines whether the function returns an array of transforms or a single transform. The (s) determines whether a single tag or all tags are used for searching. These functions return a transform or an array of transforms.

Public static Transform FindWithTag (multiple options):

- Takes either:
 - o Integer
 - o TagMask
- Returns the first transform found with any tag in the mask.

Public static Transform FindWithTags (multiple options):

- Takes either:
 - o Integer
 - o TagMask
- Returns the first transform found with all the tags in the mask (no different tag).

Public static Transform[] FindAllWithTag (multiple options):

- Takes either:
 - o Integer
 - o TagMask
- Returns all the transforms found with any of the tags in the mask.

Public static Transform[] FindAllWithTags (multiple options):

- Takes either:
 - o Integer
 - o TagMask
- Returns all the transforms found with all of the tags in the mask (no different tag).

3.2.8 Updating TagMask Functions

These functions are responsible for adding, removing or toggling a tag in the mask. The naming rules are:

- AddTag(s)ToMask
- RemoveTag(s)FromMask
- ToggleTag(s)FromMask

Where the (s) determines whether a single or multiple tags will be used while updating the mask. These functions return a TagMask.

Public static TagMask AddTagToMask (string):

- Adds a tag and returns the updated TagMask.

Public static TagMask AddTagsToMask (multiple options):

- Takes either:
 - o String[]
 - o List<string>
- Adds multiple tags and returns the updated TagMask.

Public static TagMask RemoveTagFromMask (string):

- Remove a tag and returns the updated TagMask.

Public static TagMask RemoveTagsFromMask (multiple options):

- Takes either:
 - o String[]
 - o List<string>
- Removes multiple tags and returns the updated TagMask.

Public static TagMask ToggleTag (string):

- toggles a tag and returns the updated TagMask.

Public static TagMask ToggleTags (multiple options):

- Takes either:
 - o String[]
 - o List<string>
- Toggles multiple tags and returns the updated TagMask.

3.2.9 Finding Game Objects and Transforms without MultipleTags

All of the finding-based functions (like FindGameObjectsWithTags) require [MultipleTags](#) component on your game objects to be able to work. However, these next functions do not require [MultipleTags](#) attached game objects, but they still utilize [TagMask](#) if you want to use [TagMask](#) functionality with the default Unity tag system.

For example, suppose you want to find all game objects that you tagged using the default Unity tag system, but you also want to include game objects with different specific tags in your search. Instead of creating multiple lists and combining them, you can use these functions to include [TagMask](#) in your search.

Naming rules for these functions are as follows:

- FindGameObject(s)WithoutMultipleTags
- Find(All)WithoutMultipleTags

Where (s) and (All) determines whether a single or all of the game objects or transforms found are going to be returned by the functions.

FindGameObjectWithoutMultipleTags (multiple options)

- Takes either:
 - o Integer
 - o TagMask
- Returns the first found game object tagged with any of the tags in the mask.

FindGameObjectsWithoutMultipleTags[] (multiple options)

- Takes either:
 - o Integer
 - o TagMask
- Returns all found game objects tagged with any of the tags in the mask.

FindWithoutMultipleTags (multiple options)

- Takes either:
 - Integer
 - TagMask
- Returns the first found transform tagged with any of the tags in the mask.

FindAllWithoutMultipleTags[] (multiple options)

- Takes either:
 - Integer
 - TagMask
- Returns all found transforms tagged with any of the tags in the mask.

5: References

For more information, check out the [YouTube Channel](#).

[1] Unity Technologies (2024) LayerMask, Unity3d.com. [Online]. Available at: <https://docs.unity3d.com/ScriptReference/LayerMask.html>.